

EVENT STREAMING PLATFORM

Apache Kafka

An introduction to distributed event streaming, for engineers who haven't met it yet.

What Is Kafka?

Apache Kafka is a distributed platform for publishing, storing, and processing **streams of events** in real time.

Distributed

Durable

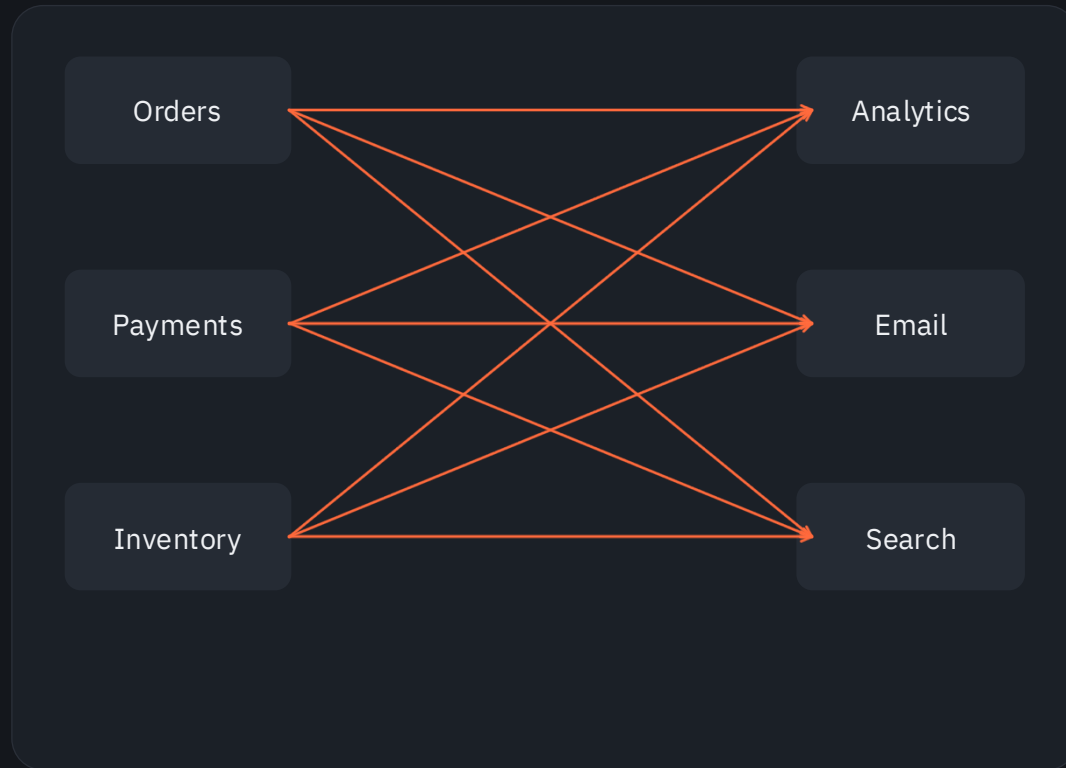
Real-Time

Scalable

Originally built at LinkedIn in 2011, Kafka is now the de facto standard for moving data between systems at high volume and low latency.

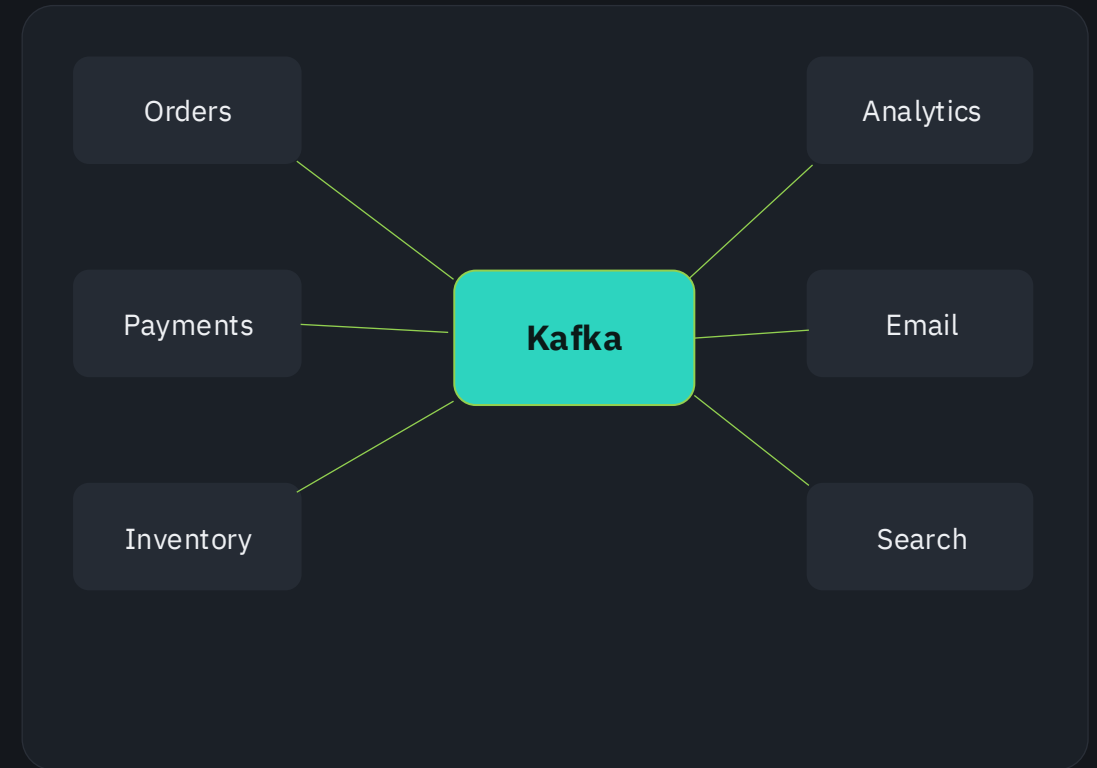
The Problem: Everyone Talks to Everyone

Without Kafka



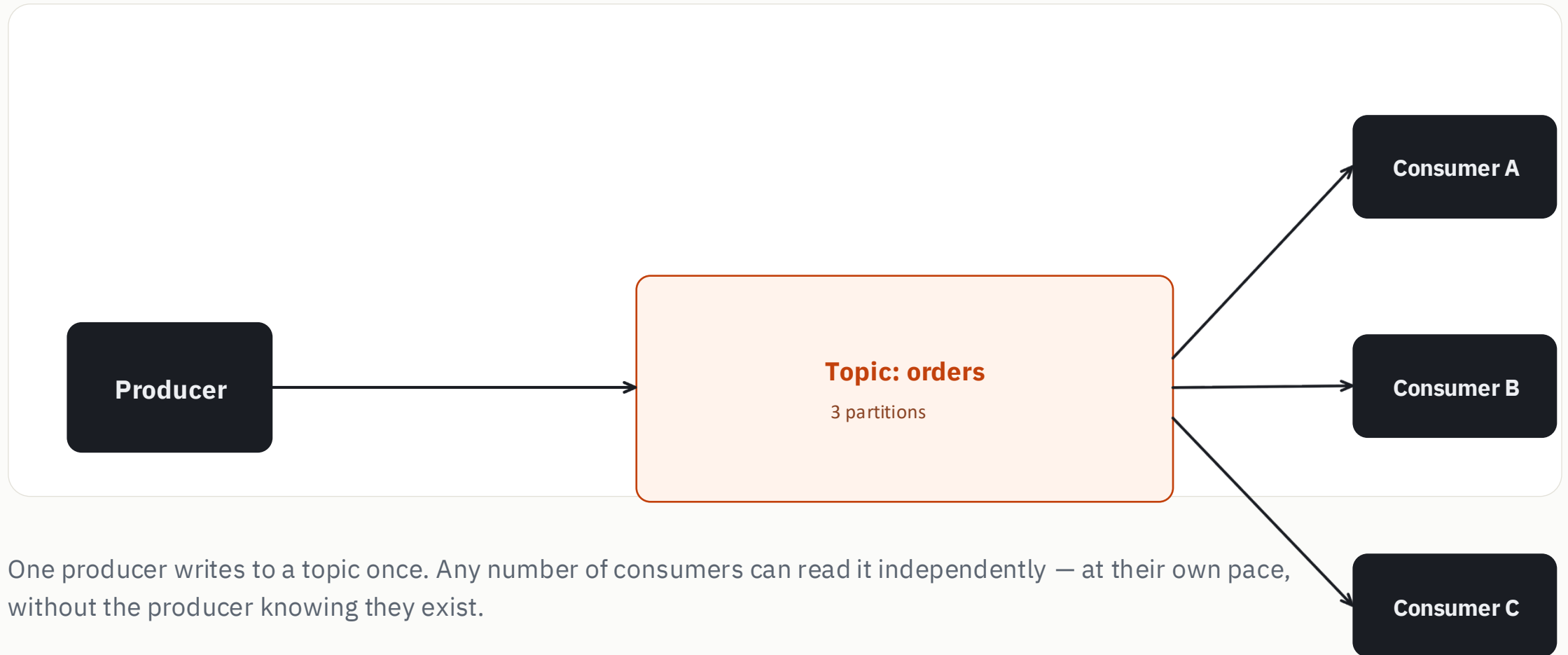
N systems need up to N^2 point-to-point connections.

With Kafka



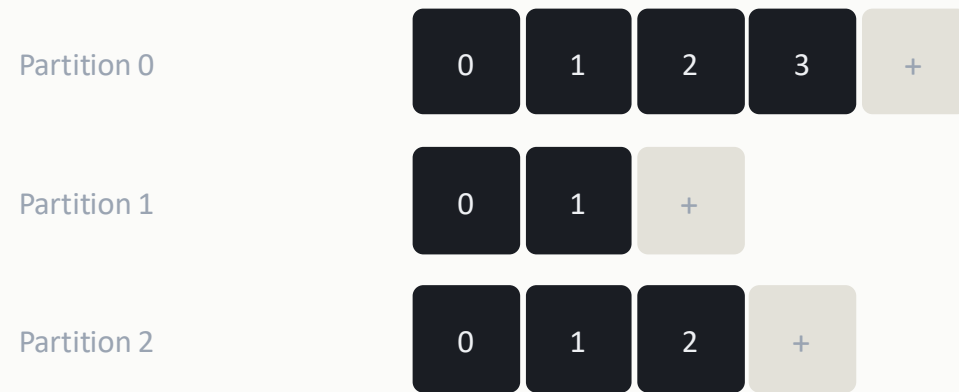
Every system just talks to Kafka.

Publish / Subscribe, Not Point-to-Point



Topics Are Split Into Partitions

A **topic** is a named stream of events, e.g. **orders**. Kafka splits it into **partitions** — ordered, append-only logs identified by an offset.



Order is only guaranteed **within** a partition — never across the whole topic.

Producers Write. Consumers Read.



Producers

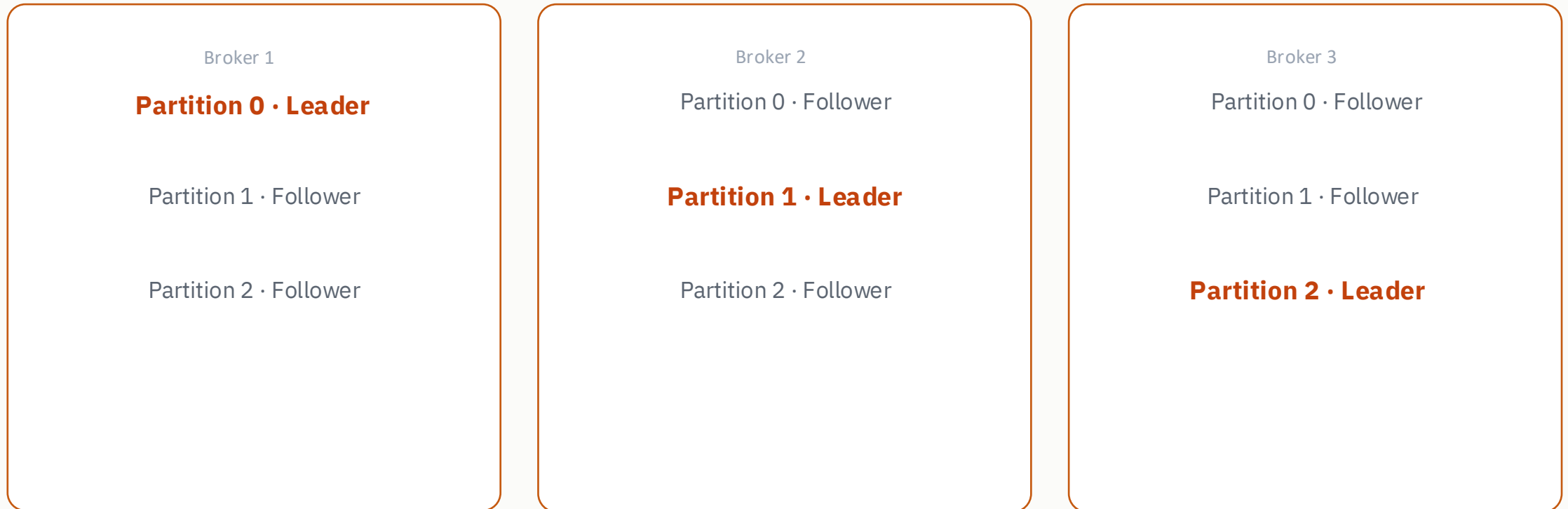
- Applications that publish events to a topic
- Can choose (or hash) a key to pick a partition; defaults to round-robin delivery
- Don't know or care who reads the data



Consumers

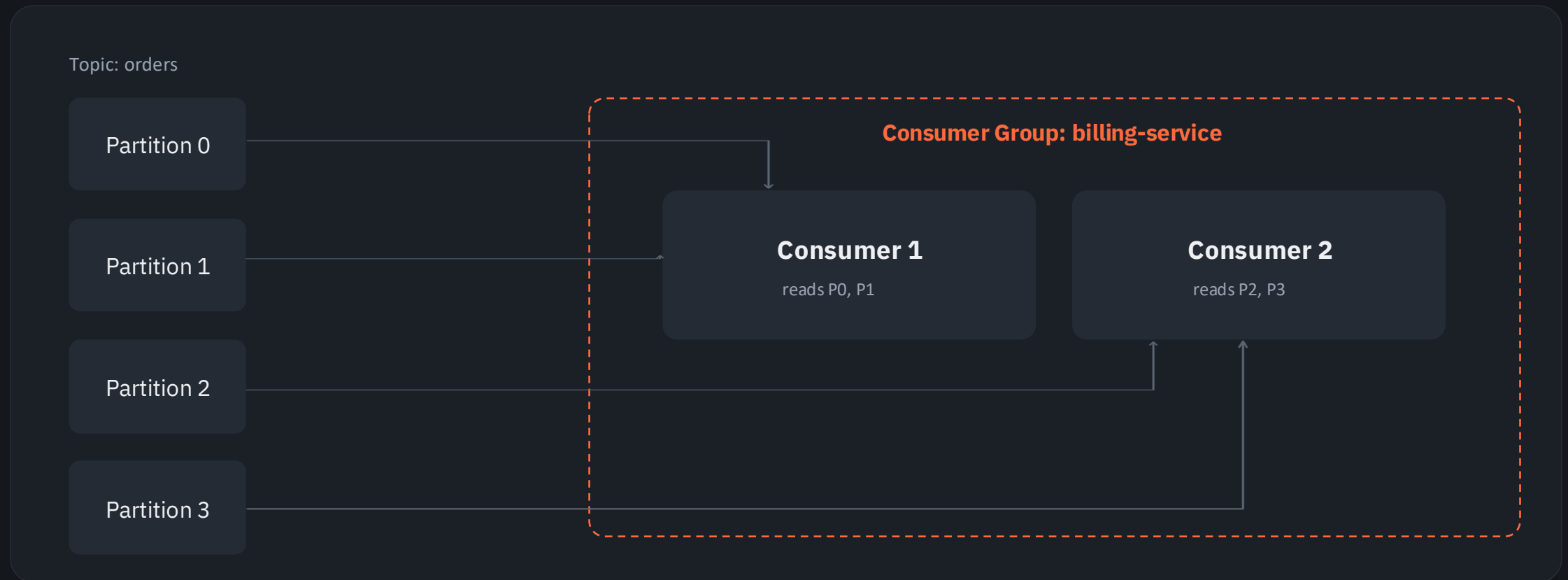
- Applications that subscribe and read events
- Track their own position with an **offset**
- Can replay history by resetting that offset

Brokers Form the Cluster



Each partition has one **leader** broker that handles reads and writes, and replicas on other brokers for fault tolerance. If a broker fails, a follower takes over.

Consumer Groups Scale Reads



Each partition is read by exactly one consumer in a group — that's how client apps can scale beyond a single consumer

Who Deletes the Message, and When?

The biggest difference between Kafka and a traditional queue (RabbitMQ, ActiveMQ, JMS) is how — and when — a message goes away.



Traditional Queue

- The broker must track **every subscriber** and what each one has already acknowledged
- A message is deleted only once **all** subscribers have consumed it
- Add a subscriber, and the broker takes on more per-message bookkeeping



Kafka

- The broker does **not** track which consumers have read a message
- Every message simply stays until its **retention period** expires (e.g. 7 days)
- Each consumer tracks its own offset — the broker's job never gets more complex

Traditional queues: bookkeeping grows with the number of subscribers. **Kafka:** one retention clock per topic, no matter how many consumers are reading.

Why Kafka Holds Up at Scale

Property	What it means
Durability	Events are written to disk and replicated across brokers before being acknowledged
Scalability	Add brokers and partitions to handle more throughput, with no downtime
Fault tolerance	Replica brokers take over automatically if a leader fails
Ordering	Guaranteed within a partition, so events for the same key stay in sequence
High throughput	Sequential disk I/O and batching let a single broker handle millions of events/sec

Where Kafka Shows Up



Real-time analytics

Dashboards and fraud detection on live data



Log aggregation

Centralizing logs from thousands of services



Microservice events

Decoupled communication between services



Metrics & monitoring

Streaming system and application metrics



Data integration

Feeding data lakes, warehouses, and search indexes



Event sourcing

The log as the source of truth for state changes

Beyond the Core: the Ecosystem

Tool	Purpose
Kafka Connect	Prebuilt connectors to import/export data (databases, S3, Elasticsearch...)
Kafka Streams	A Java library for building stream-processing applications
ksqlDB	SQL-like queries over streaming data, no code required
Schema Registry	Enforces and versions the structure of messages over time

The Full Picture



Key Takeaways

- ✓ Kafka is a **distributed event streaming platform**, not a traditional message queue
- ✓ Topics are split into ordered **partitions** — the unit of parallelism
- ✓ **Producers** and **consumers** are fully decoupled — consumers control their own offset
- ✓ **Consumer groups** parallelize reads, each partition to only one consumer
- ✓ Replication across **brokers** gives Kafka durability and fault tolerance

THANK YOU

Questions?